

Spring Boot MCQ Set 1 May 2026 Cycle

Question 1

Your Tech Lead assigns you a high-priority bug ticket for a newly deployed e-commerce microservices architecture. The system uses a stateless Spring Security configuration with JWTs signed using an RSA key pair. Users are reporting that immediately after clicking "Logout" on one browser tab, they can switch to another tab with the same session open and successfully submit a purchase order. The TL emphasizes that the logout must be instantaneous across all services, but we strictly cannot change the stateless, decentralized nature of the resource servers (they must continue validating tokens locally via the public key without hitting an authorization server on every request).

What is the most architecturally sound approach to implement this instantaneous logout requirement in your Spring Boot security configuration?

- **A)** Modify the security filter chain to intercept the logout request and immediately destroy the SecurityContextHolder across the cluster. Then, shorten the JWT expiration time in the authorization server to 30 seconds to minimize the vulnerability window, relying on the client to silently refresh tokens in the background.
- **B)** Implement an in-memory or Redis-backed "denylist" (blacklist) at the API Gateway level. When a user logs out, the JWT's unique identifier (jti claim) is added to this denylist with a Time-To-Live (TTL) matching the token's remaining validity. A custom OncePerRequestFilter on the resource servers will check this denylist before authenticating the request.
- **C)** Transition the SessionCreationPolicy in the SecurityFilterChain from STATELESS to IF_REQUIRED. This allows Spring Security to maintain a lightweight session ID mapped to the JWT, so when the /logout endpoint is hit, the session handler can explicitly invalidate the JSESSIONID cookie and clear the security context simultaneously.
- **D)** Intercept the /logout request and trigger a Spring Application Event that broadcasts a message via a message broker (like RabbitMQ) to all running instances of the microservices. Each instance will then receive the event and remove the corresponding user's Authentication object from their local JVM memory heap.

Answer: B

Explanation: In a purely stateless JWT architecture, the server does not track sessions. Once a token is issued, it is valid until it expires. You cannot "destroy" a JWT remotely. Option B is the industry-standard pattern for revoking stateless tokens; the denylist only needs to hold the token until its natural expiration, keeping memory usage low while enforcing immediate revocation. Options A and D are ineffective or overly complex workarounds, and Option C violates the core requirement of maintaining a stateless architecture.

Question 2

You are developing a notification service that sends a daily summary email to users. You implemented a method annotated with @Scheduled(cron = "0 0 8 * * ?") to run at 8:00 AM daily. It works perfectly in your local machine and the single-instance staging environment. However, three days after deploying to the production environment—which consists of three load-balanced Kubernetes pods—customer support is flooded with tickets. Users are complaining that they are receiving exactly three identical summary emails at 8:00 AM every single day.

What is the most robust, standard way within the Spring ecosystem to resolve this issue without hardcoding instance IDs or fundamentally changing the deployment architecture?

- **A)** Inject the Environment bean into your scheduled class and check the `spring.application.instance_id` property. Wrap the execution logic in an if statement that only allows the instance with an ID ending in "1" or "0" to execute the mailing logic, ignoring it on the others.
- **B)** Remove the `@Scheduled` annotation and instead configure a `TaskExecutor` bean with a thread pool size of 1. Because the load balancer sits in front of the application, the single thread pool will guarantee that only one incoming request or internal trigger can process the notification batch at any given time.
- **C)** Integrate a distributed locking library, such as `ShedLock`, and annotate the method with `@SchedulerLock`. Configure the lock provider to use your existing shared relational database or Redis cache. This ensures that the first pod to wake up at 8:00 AM acquires the lock, and the other pods will skip the execution.
- **D)** Change the cron expression dynamically using `@ConditionalOnProperty`. Create a configuration class that assigns a slightly different cron schedule to each pod during startup (e.g., Pod 1 runs at 8:00 AM, Pod 2 at 8:01 AM). Then, write a shared database flag indicating if the day's job has already run.

Answer: C

Explanation: Spring's `@Scheduled` annotation runs locally on every JVM instance where it is deployed, leading to duplicate executions in a cluster. Option C (using a library like `ShedLock` or `Quartz`) is the correct architectural pattern, using an external, shared state (DB/Redis) to guarantee single execution across a cluster. Options A and D are brittle, manual workarounds. Option B misunderstands how thread pools work across distributed systems.

Question 3

During a Friday afternoon code review, the Database Administrator flags a massive spike in database I/O originating from your newly deployed `InventoryReportingService`. You investigate and trace it to a method executing `orderRepository.findAll()`, which retrieves exactly 1,000 recent orders. Immediately after, a mapping layer iterates through these orders, calling `order.getCustomer().getEmail()` to build a report. The Customer entity is mapped inside the Order entity using `@ManyToOne(fetch = FetchType.LAZY)`. The DBA confirms that 1,001 separate SQL queries are hitting the database sequentially.

Which of the following is the most efficient Spring Data JPA solution to resolve this specific bottleneck, ensuring only one query is executed, while strictly preserving the lazy-loading strategy for all other parts of the application?

- **A)** Modify the entity mapping in the Order class to `@ManyToOne(fetch = FetchType.EAGER)`. Because `findAll()` is an intrinsic Spring Data method, changing the fetch type at the entity level is the only way to force Hibernate to perform a SQL JOIN automatically without writing custom queries.
- **B)** Leave the entity mapping as LAZY. In the `OrderRepository`, create a custom method `List<Order> findAllOrdersWithCustomers()` and annotate it with `@Query("SELECT o FROM Order o JOIN FETCH o.customer")`. Update the reporting service to call this new method instead of the standard `findAll()`.
- **C)** Wrap the reporting method in a `@Transactional(readOnly = true)` annotation and call `Hibernate.initialize(order.getCustomer())` inside the loop. The read-only transaction context will automatically batch the initialization calls into a single background query before the loop finishes.

- **D)** Fetch all orders using `findAll()`, and then separately fetch all customers using `customerRepository.findAllByIdIn(orderIds)`. Finally, use Java Streams to manually map and group the customers back into the order objects in application memory to bypass Hibernate's proxy initialization.

Answer: B

Explanation: This is the classic N+1 query problem. Option B uses JOIN FETCH, which translates to a SQL INNER JOIN, fetching both entities in a single query while leaving the global entity configuration as LAZY. Option A is incorrect because changing to EAGER will impact the entire application and often still results in N+1. Option C does not batch queries by default. Option D works but is a manual anti-pattern when the ORM can handle it directly.

Question 4

Your team is building a complex B2B portal. You create a `DocumentProcessingService` annotated with `@Service`. To optimize performance and avoid recreating heavy XML parsers, you define a class-level variable: `private List<String> parsedErrors = new ArrayList<>();`. Inside your `processDocument(File file)` method, you add any validation failures to this list, and at the end of the method, you return the list to the controller so the user sees their specific errors. During concurrent load testing, users start seeing validation errors belonging to entirely different companies' documents.

What fundamental Spring Core concept is being violated here, and what is the standard, thread-safe fix for this specific architecture?

- **A)** The `DocumentProcessingService` is registered as a Prototype bean by default in Spring Boot. Therefore, multiple threads are trying to instantiate the bean simultaneously, causing a race condition on the `ArrayList` allocation. The fix is to annotate it with `@Scope("singleton")`.
- **B)** The `ArrayList` implementation is not thread-safe. When multiple HTTP requests hit the service concurrently, the JVM throws a `ConcurrentModificationException` internally, which Spring catches and swallows, leading to corrupted data states. The fix is to change it to `CopyOnWriteArrayList`.
- **C)** Spring `@Service` beans are Singletons by default. Only one instance of `DocumentProcessingService` exists in the entire JVM. Therefore, the `parsedErrors` list is a globally shared state across all concurrent HTTP threads. The fix is to remove the class-level state and instantiate the `List` locally inside the `processDocument` method.
- **D)** The Controller calling the service is operating in a Request scope, but the Service is in a Session scope. The errors are bleeding across users because the session ID is not being properly passed in the HTTP headers from the load tester. The fix is to inject the `HttpServletRequest` into the service to isolate the data.

Answer: C

Explanation: By default, all Spring beans are Singletons, meaning multiple concurrent threads execute the methods of the exact same object instance. Defining mutable state (like a `List`) at the class level means every thread shares and modifies that same list, causing data to bleed across requests. Option C identifies the root cause and provides the correct best practice: building stateless services.

Question 5

You are implementing a user registration flow. Your `registerNewUser` method in the `@Service` layer is annotated with `@Transactional`. Inside this method, you first save the `User` entity to the database using JPA. Then, on the next line, you call an external, legacy SOAP API to provision a workspace. The legacy client signature explicitly declares throws `WorkspaceProvisioningException`, which is a custom exception extending `java.lang.Exception`. During testing, the SOAP API goes down, and the method correctly throws the `WorkspaceProvisioningException`. However, you notice the `User` record was still permanently saved to the database; the transaction did not roll back.

Why did Spring fail to roll back the database transaction, and how should you properly configure the code to ensure strict atomicity in this scenario?

- **A)** Spring's `@Transactional` proxy cannot intercept exceptions thrown by external network calls; it only monitors exceptions originating from the underlying `DataSource` or `JPA EntityManager`. You must manually call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`.
- **B)** By default, Spring's declarative transaction management only triggers a rollback for unchecked exceptions (`RuntimeException` and its subclasses) and `Error`. Because `WorkspaceProvisioningException` extends `Exception`, it is a checked exception. The fix is to update the annotation to `@Transactional(rollbackFor = Exception.class)`.
- **C)** The external SOAP call is blocking the thread, causing the database connection pool to timeout. When the timeout occurs, Hibernate automatically issues a `COMMIT` statement to prevent table locking before the exception propagates up the stack. The fix is to make the API call asynchronous using `@Async`.
- **D)** The method is likely private or package-private. Spring AOP uses dynamic proxies that can only intercept and apply transactional advice to public methods. The fix is to change the method visibility to public and ensure it is called from an external bean.

Answer: B

Explanation: By default, `@Transactional` only rolls back on unchecked exceptions (`RuntimeException`). If a checked exception (extending `java.lang.Exception`) is thrown, Spring assumes it is a recoverable business error and commits the transaction. Updating the annotation to explicitly rollback for `Exception.class` (Option B) is the correct fix.

Question 6

You are optimizing a high-traffic Spring Boot microservice. To reduce database calls, you implement a local `HashMap<UserRole, List<Permissions>>` to cache access rights. The `UserRole` object has two fields: `String roleName` and `int accessLevel`. After retrieving a `UserRole` from the DB and inserting it into the map as a key, a downstream service updates the `accessLevel` integer on that exact `UserRole` object instance in memory. Five minutes later, an incoming request attempts to retrieve the permissions using that same `UserRole` object, but `cacheMap.get(userRole)` returns null, causing a `NullPointerException`.

Why did the `HashMap` fail to retrieve the value using the exact same object instance?

- **A)** `HashMap` is not thread-safe. A concurrent garbage collection cycle likely triggered a rehash of the internal buckets, temporarily invalidating the memory reference of the `UserRole` object.
- **B)** Modifying a field that is used in the `hashCode()` calculation after the object is used as a key in a Hash-based collection violates the Hash contract. The map looked in the bucket corresponding to the *new* hash code, but the entry was stored in the bucket for the *old* hash code.

- **C)** The equals() method inside the UserRole class was not overridden. Even though it is the exact same object in memory, HashMap strictly requires a custom equals() implementation to resolve hash collisions during the get() operation.
- **D)** Spring's internal caching proxy automatically clears in-memory standard Java Collections after a default timeout of 60 seconds unless they are wrapped in a @Cacheable annotation manager.

Answer: B

Explanation: This is a classic violation of the Java Collections Framework contract. If an object is used as a key in a HashMap or HashSet, its state—specifically the fields used to calculate its hashCode()—must remain immutable. By changing the accessLevel, the hash code changed. When get() was called, the map calculated the new hash, looked in the wrong internal bucket, and found nothing.

Question 7

You are building a multithreaded order processing engine. You have a central MetricsTracker class with a private volatile int totalOrdersProcessed = 0;. You have 50 worker threads that concurrently pick up messages from a queue and, upon success, execute metricsTracker.increment(), which contains the single line of code: totalOrdersProcessed++;. At the end of a stress test where exactly 10,000 orders were successfully processed, the database shows 10,000 new rows, but the totalOrdersProcessed variable outputs a random number like 9,842.

Why is the count inaccurate, and what is the most performant Core Java solution?

- **A)** The volatile keyword forces threads to read from main memory, but it does not guarantee atomicity for compound operations. totalOrdersProcessed++ is actually three operations (read, add, write). Multiple threads are reading the same value before any of them write the updated value. The fix is to use an AtomicInteger and its incrementAndGet() method.
- **B)** The volatile keyword is designed for object references, not primitive types. Because int is a primitive, the JVM optimizes it by placing it in the thread-local CPU L1 cache, ignoring the volatile directive. The fix is to change it to volatile Integer totalOrdersProcessed.
- **C)** The worker threads are throwing unchecked exceptions that are being swallowed by the message queue's default error handler, causing the increment line to be skipped. The fix is to wrap the increment logic in a finally block.
- **D)** 50 threads exceed the default OS thread scheduler limits for concurrent memory access. The JVM is randomly dropping instructions to prevent an OutOfMemoryError. The fix is to wrap the increment line in a synchronized(this) block.

Answer: A

Explanation: The volatile keyword ensures *visibility* (threads always see the most recent write) but does not ensure *atomicity*. The post-increment operator (++) is a compound operation. If two threads read the value '10' simultaneously, they both increment to '11' and write '11' back, losing an increment. AtomicInteger uses low-level Compare-And-Swap (CAS) CPU instructions to ensure thread-safe, lock-free atomicity.

Question 8

You are refactoring a legacy banking application. You have an interface TransactionValidator with a method: void validate(Transaction t) throws IOException;. You create a new implementation called InternationalTransactionValidator. The business requirements state

that this new implementation must throw a custom `SanctionsListException` (which extends `java.lang.Exception`) if the transaction originates from a blocked country, and a `IllegalArgumentException` if the transaction amount is negative.

When you attempt to compile the new `InternationalTransactionValidator` class overriding the `validate` method, what will the Java compiler do?

- **A)** It will compile successfully, provided that the calling class catches both `IOException` and `Exception`.
- **B)** It will fail to compile because an overriding method cannot throw *new* or *broader* checked exceptions than the overridden method. `SanctionsListException` is a checked exception not declared in the interface.
- **C)** It will fail to compile because `IllegalArgumentException` is not declared in the interface signature, and interfaces strictly dictate all exceptions that implementation methods are allowed to throw.
- **D)** It will compile successfully because `SanctionsListException` extends `Exception`, making it a subclass of Java's universal `Throwable` hierarchy, which is implicitly accepted by all interface contracts.

Answer: B

Explanation: Java enforces strict rules on method overriding regarding exceptions. An overriding method cannot throw new *checked* exceptions, nor can it throw broader checked exceptions than the parent method. Because `SanctionsListException` extends `Exception`, it is a checked exception. `IllegalArgumentException`, however, is a `RuntimeException` (unchecked), which can be thrown freely without being declared in the signature.

Question 9

Your DBA creates a new business rule in your Oracle database: "No employee in the `EMPLOYEES` table can have a salary greater than the average salary of their respective department." To enforce this, a junior developer writes a `BEFORE UPDATE` row-level trigger on the `EMPLOYEES` table. Inside the trigger, they execute a `SELECT AVG(salary) FROM EMPLOYEES WHERE department_id = :NEW.department_id` and raise an application error if `:NEW.salary` exceeds this average. When testing, attempting to update an employee's salary throws an `ORA-04091: table EMPLOYEES is mutating, trigger/function may not see it` error.

What is the architectural reason for this error, and how should this logic actually be implemented?

- **A)** Row-level triggers fire before the data is committed. Therefore, the `AVG` function returns a `NULL` value, causing a division-by-zero error internally that Oracle masks as a mutating table error. The fix is to change it to an `AFTER UPDATE` trigger.
- **B)** A row-level trigger cannot query the same table that is currently undergoing DML (Data Manipulation Language) changes because the table is in an inconsistent state (mutating). The proper architectural fix is to enforce this complex cross-row validation at the Application Service layer (Java), or use a Compound Trigger to aggregate data at the statement level.
- **C)** The `:NEW` pseudo-record is only available in `INSERT` triggers, not `UPDATE` triggers. The database engine does not know which department ID to query. The fix is to use the `:OLD.department_id` instead.
- **D)** The trigger requires explicit transaction control. Because it is doing a `SELECT` during an `UPDATE`, it needs to issue a `COMMIT` statement immediately before the `SELECT AVG()` to ensure read consistency.

Answer: B

Explanation: The Mutating Table error (ORA-04091) is a classic Oracle safeguard. While a row is being updated, the table is in flux. If a row-level trigger tries to query that same table, it cannot guarantee a consistent read. You cannot read from the table you are actively modifying in a row-level trigger. The best practice is to move complex business logic like this to the application layer (Spring Boot) or use modern PL/SQL Compound Triggers if it must stay in the DB.

Question 10

A nightly batch job processes 5 million archival records from STAGING_ORDERS and inserts them into HISTORY_ORDERS. The current PL/SQL procedure uses a standard explicit cursor: `FOR rec IN (SELECT * FROM STAGING_ORDERS) LOOP INSERT INTO HISTORY_ORDERS VALUES rec; END LOOP;` The job is taking 4 hours to run. The system monitoring tools show that the CPU is entirely consumed by "Context Switches" between the PL/SQL engine and the SQL engine.

Which Oracle PL/SQL feature should you implement to drastically reduce this context switching and optimize the batch insertion?

- **A)** Add the `PRAGMA AUTONOMOUS_TRANSACTION` directive to the procedure. This runs the loop in a separate memory space, bypassing the SQL engine's redo log bottlenecks.
- **B)** Replace the standard `FOR` loop with `BULK COLLECT INTO` to fetch records into a PL/SQL collection (array) in batches (e.g., using a `LIMIT` of 1000), and then use the `FORALL` statement to push the batch of inserts to the SQL engine in a single sweep.
- **C)** Wrap the `INSERT` statement in an `EXECUTE IMMEDIATE` block to force dynamic SQL execution, which bypasses the PL/SQL parser and executes directly on the SQL engine.
- **D)** Disable all primary keys and indexes on the `STAGING_ORDERS` table before the loop begins, and rebuild them after the loop completes using `ALTER INDEX ... REBUILD`.

Answer: B

Explanation: In PL/SQL, loops that execute DML (Insert/Update/Delete) row-by-row cause massive performance overhead because the execution control jumps back and forth (context switches) between the PL/SQL engine (processing the loop) and the SQL engine (executing the insert) for every single row. `BULK COLLECT` and `FORALL` allow you to bundle arrays of data and send them to the SQL engine in one massive chunk, drastically reducing context switches and execution time.

Question 11

You are building a `/api/v1/orders` endpoint that accepts a massive JSON payload. The controller method signature is `public ResponseEntity<?> createOrder(@Valid @RequestBody OrderDTO orderDto)`. The `OrderDTO` contains basic fields like `customerId`, which validate perfectly using `@NotBlank`. It also contains a `List<OrderItemDTO> items`. You placed `@Positive` on the `price` field inside the `OrderItemDTO` class. During integration testing, a QA engineer successfully submits an order where one of the items has a price of `-500.00`, bypassing your validation entirely and causing negative revenue in the database.

Why did the Spring Validator ignore the annotations inside the nested `OrderItemDTO` list, and how do you fix it?

- **A)** The `@Valid` annotation only triggers validation on the top-level scalar fields of the immediate object. To validate nested collections, you must write a custom Spring Validator class and manually register it in an `@InitBinder` method within the controller.
- **B)** Spring Boot's default validation provider (Hibernate Validator) does not cascade validation to complex object graphs or collections automatically to prevent infinite recursion and performance bottlenecks. You must add the `@Valid` annotation directly to the `List<OrderItemDTO> items` field inside the `OrderDTO` class.
- **C)** Collections like `List` or `Set` cannot be directly validated by the standard Jakarta Validation API. You must wrap the list in a custom wrapper class (e.g., `OrderItemCollection`) and place the validation annotations on that wrapper.
- **D)** The `@Positive` annotation is strictly for primitive numeric types (`int`, `double`). If the `price` field in `OrderItemDTO` is a `BigDecimal` or a wrapper class like `Double`, the validation is silently ignored. You must use `@DecimalMin(value = "0.01")` instead.

Answer: B

Explanation: By default, `@Valid` only checks the immediate fields of the object it annotates. When it encounters a nested object or a collection of objects, it does not automatically cascade into them. You must explicitly tell the validator to cascade by placing `@Valid` on the nested field declaration itself (e.g., `@Valid private List<OrderItemDTO> items;`). Option D is false; `@Positive` fully supports `BigDecimal`.

Question 12

You have built a robust, global exception handler using `@RestControllerAdvice`. It catches `SignatureException` (thrown when a JWT is invalid) and returns a beautifully formatted JSON response with a custom error code and a 401 status. However, the frontend team reports that when they send an expired or mathematically invalid JWT, they do not get your JSON response. Instead, they receive a generic HTML error page generated by Apache Tomcat, breaking their JSON parser.

Why is your `@RestControllerAdvice` failing to catch this specific exception?

- **A)** The JWT validation logic is likely executing inside a Spring Security `OncePerRequestFilter`. Filters sit in front of the `DispatcherServlet`. `@RestControllerAdvice` only intercepts exceptions thrown *after* the request has reached the `DispatcherServlet` (e.g., in Controllers or Services).
- **B)** Exceptions related to security or authentication are strictly handled by the `AccessDeniedHandler`, which takes precedence over `@RestControllerAdvice`. You must explicitly disable the default Spring Security exception translator in your `SecurityFilterChain`.
- **C)** The `@RestControllerAdvice` is missing the `basePackages` attribute. By default, it only scans the exact package it resides in. If the security filter is in a different package, the advice cannot detect exceptions originating from it.
- **D)** The exception handler method is returning a `ResponseEntity<Object>`, but the HTTP `Accept` header from the frontend is missing, causing Spring's content negotiation to fallback to the default HTML error view instead of serializing the JSON.

Answer: A

Explanation: The `DispatcherServlet` acts as the Front Controller for MVC. `@RestControllerAdvice` is an AOP mechanism that wraps the `DispatcherServlet`. Security filters (like a custom JWT filter) are executed in the application server's Filter Chain *before* the request ever reaches the `DispatcherServlet`. Therefore, exceptions thrown in filters bubble up to the servlet container (Tomcat), bypassing your MVC advice entirely.

The standard fix is to use a `HandlerExceptionResolver` inside the filter to forward the error to the advice.

Question 13

You are debugging a database connection issue in a newly deployed staging environment. Your application has an `application.yml` (containing local H2 DB credentials) and an `application-staging.yml` (containing the correct AWS RDS credentials). The DevOps team confirms the application is starting with the environment variable `SPRING_PROFILES_ACTIVE=staging`. However, the application crashes on startup because it is trying to connect to a legacy on-premise database. You search the codebase and find that a senior developer recently added a class annotated with `@Configuration` and `@PropertySource("classpath:legacy-database.properties")` to support an older batch job, and this file contains the legacy DB credentials.

According to Spring Boot's externalized configuration resolution order, why is the application prioritizing the legacy properties over the profile-specific YAML file?

- **A)** Properties defined via `@PropertySource` annotations always have absolute top priority over any `application.yml` or `application-{profile}.yml` files, regardless of active profiles, because they are evaluated explicitly during the bean definition phase.
- **B)** Actually, profile-specific YAML files (`application-staging.yml`) have higher precedence than `@PropertySource`. The crash must be caused by the YAML file being malformed, which forces Spring to silently drop down to the next available property source.
- **C)** YAML files are loaded by the `YamlPropertySourceLoader`, which executes *after* standard `.properties` files. Because `legacy-database.properties` is a standard properties file, it overwrites the YAML configurations in the Environment context.
- **D)** `@PropertySource` does not have higher priority than profile-specific application properties. However, if the legacy properties file contains keys that exactly match the Spring datasource keys, and no `@Primary` datasource is defined, Spring merges the two, favoring the last loaded configuration which happens to be the `@PropertySource` due to component scanning order.

Answer: B

Explanation: This tests deep knowledge of Spring's property resolution order. Option A is a common misconception; `@PropertySource` actually has a very low priority in the Spring Boot environment. Profile-specific application properties (`application-{profile}.yml`) have a much higher precedence and will override `@PropertySource`. Therefore, if the legacy DB is being used, there is a structural issue (like malformed YAML or a typo in the profile name) causing the profile configuration to fail to load entirely.

Question 14

To decouple your monolithic architecture, you introduce Spring's `ApplicationEventPublisher`. When a user registers, the `UserService` publishes a `UserRegisteredEvent`. You have three separate `@Component` classes with `@EventListener` methods listening for this event: one to send a welcome email, one to provision a workspace, and one to sync the user to a CRM. During load testing, the `/register` API endpoint response time spikes from 50ms to 8,000ms.

What is the default behavior of Spring Events causing this latency, and what is the standard architectural fix?

- **A)** Spring Events are placed into an unbounded blocking queue. Under heavy load, the queue becomes a bottleneck, forcing the main thread to wait for space to free up. The fix

is to configure a `ThreadPoolTaskExecutor` bean and annotate the publisher with `@Async`.

- **B)** By default, all `@EventListener` methods execute synchronously on the exact same thread that published the event. The API response cannot be sent until the email, workspace, and CRM logic all finish executing sequentially. The fix is to add `@EnableAsync` to a configuration class and `@Async` to the listener methods.
- **C)** Events are asynchronous by default, but they share a single background daemon thread. Because the CRM sync involves a heavy network call, it blocks the single background thread, backing up the other events. The fix is to increase the `spring.task.execution.pool.max-size` property in the YAML.
- **D)** The Event Publisher uses standard Java Reflection to find listeners at runtime. Scanning the entire application context for `@EventListener` annotations on every API request causes massive CPU overhead. The fix is to register the listeners programmatically during application startup.

Answer: B

Explanation: A widespread misconception is that Spring Events (like an event bus) are inherently asynchronous. They are not. By default, `ApplicationEventPublisher.publishEvent()` blocks the calling thread, executing every listener synchronously and sequentially. To make them fire-and-forget, you must explicitly enable asynchronous processing with `@EnableAsync` and mark the listener methods with `@Async`.

Question 15

You developed a real-time collaborative dashboard using Spring WebSockets and STOMP. Clients subscribe to `/topic/board-updates`. When a change occurs, the server broadcasts it using `SimpMessagingTemplate.convertAndSend("/topic/board-updates", payload)`. The application works flawlessly in staging with one server. When deployed to production behind a load balancer with three instances (Node A, Node B, Node C), users report that they only see real-time updates from colleagues who happen to be connected to the exact same server node.

How do you solve this "split-brain" WebSocket issue in a distributed Spring Boot environment?

- **A)** Enable Sticky Sessions (Session Affinity) on the load balancer. This ensures that all WebSocket traffic from a specific company is routed to the same node, bypassing the need to sync messages across the cluster.
- **B)** Change the `convertAndSend` destination from a `/topic/` prefix to a `/queue/` prefix. Topics are strictly in-memory per JVM, whereas Queues in Spring STOMP are automatically distributed across the network interface.
- **C)** Replace the default simple, in-memory message broker with a full-featured external message broker (like RabbitMQ or ActiveMQ) and configure Spring's `StompBrokerRelay`. When Node A receives an update, it forwards it to RabbitMQ, which then broadcasts it back to Nodes B and C to send to their connected clients.
- **D)** Implement a `@Scheduled` task that synchronizes the internal `SimpUserRegistry` across all three nodes every 100 milliseconds using an internal REST API call, ensuring all nodes are aware of all active WebSockets.

Answer: C

Explanation: Spring's default WebSocket setup uses an in-memory message broker. It has no awareness of other JVMs in a cluster. If Node A sends a message to `/topic`, only clients connected to Node A receive it. Option C is the enterprise standard: integrating an

external message broker via StompBrokerRelay. The external broker handles the pub/sub mechanics across the entire cluster.

Question 16

You enable Spring Boot Actuator in your application by adding the dependency. You want to expose all endpoints to monitor the application, so you set `management.endpoints.web.exposure.include=*` in your YAML. To secure these sensitive endpoints, you add Spring Security to your project. You write a `SecurityFilterChain` that explicitly permits all traffic to the API: `http.authorizeHttpRequests(auth -> auth.requestMatchers("/api/**").permitAll().anyRequest().authenticated());` However, penetration testers report that they can access `/actuator/env` and `/actuator/threaddump` without any authentication.

Why are the Actuator endpoints exposed despite the `anyRequest().authenticated()` rule?

- **A)** Actuator endpoints are mapped to a completely different internal Tomcat port by default to ensure they remain accessible during traffic spikes. Because Spring Security only intercepts traffic on the primary server port (e.g., 8080), it bypasses the security filter chain entirely.
- **B)** Actuator auto-configuration automatically registers a secondary `SecurityFilterChain` bean with an `@Order` value that precedes your custom security chain. This default chain explicitly permits access to actuator endpoints unless you configure the `management.endpoint.health.roles` property.
- **C)** `anyRequest().authenticated()` only applies to custom controllers annotated with `@RequestMapping` or `@RestController`. Because Actuator endpoints are implemented using native Servlet APIs under the hood, they fall outside the scope of MVC-based security matchers.
- **D)** Actually, the provided security configuration *should* secure the actuator endpoints. If they are accessible, it means the `spring-boot-starter-security` dependency is missing from the `pom.xml`, or the `@EnableWebSecurity` annotation was omitted from the configuration class.

Answer: B

Explanation: In modern Spring Boot, Actuator can auto-configure its own security rules that may conflict with or override yours depending on the Spring Boot version and configuration. To properly secure actuator endpoints independently, developers should use `EndpointRequest.toAnyEndpoint()` in a dedicated `SecurityFilterChain` with a higher `@Order` to explicitly define how management routes are secured, rather than relying on a catch-all `anyRequest()`.

Question 17

You are generating a new Spring Boot project. In the `pom.xml`, you notice the `<parent>` tag is set to `spring-boot-starter-parent`. You need to add a dependency for Gson to parse some legacy JSON. You add `<dependency><groupId>com.google.code.gson</groupId><artifactId>gson</artifactId></dependency>`. Your junior developer flags this in a PR review, stating that you forgot to include the `<version>` tag, and the build will fail. You explain that it will compile perfectly.

Why does Maven not require a version tag for this specific dependency in a Spring Boot project?

- **A)** Maven automatically queries the Maven Central Repository API during the build process and dynamically downloads the latest stable release for any dependency that omits a version tag.
- **B)** The spring-boot-starter-parent acts as a Bill of Materials (BOM). It contains a deeply nested <dependencyManagement> section that pre-defines the optimal, tested versions for hundreds of common third-party libraries, including Gson.
- **C)** Gson is a core module included inside the standard Java Development Kit (JDK) starting from Java 17. Maven ignores the dependency tag entirely and links directly to the java.util.gson package provided by the environment.
- **D)** Spring Boot's internal classloader intercepts the missing version at runtime and utilizes a bundled, shaded version of Gson that exists inside the spring-core.jar, rendering the explicit Maven version unnecessary.

Answer: B

Explanation: By inheriting from spring-boot-starter-parent, your project inherits a massive <dependencyManagement> block. This block acts as a dictionary of guaranteed-compatible versions for hundreds of libraries. If you declare the dependency without a version, Maven looks it up in the parent's dictionary, preventing "dependency hell" caused by mismatched transitive versions.

Question 18

You are tasked with upgrading the security of a Spring Boot REST API. The current system uses long-lived JWTs. You implement a "Refresh Token Rotation" architecture. The flow is: User logs in -> gets short-lived Access Token (15 min) and long-lived Refresh Token (7 days). When the Access Token expires, the client sends the Refresh Token to /api/auth/refresh. The server validates it, generates a *new* Access Token and a *new* Refresh Token, and invalidates the old Refresh Token in the database. A user on a train goes through a tunnel just as their app requests a refresh. The server processes the request, issues the new tokens, and invalidates the old one, but the client loses network connection before receiving the response. The client app immediately retries the request using the same, original Refresh Token.

What is the strictly required security behavior the server must execute upon receiving this second request with the recently invalidated Refresh Token?

- **A)** The server should detect a network retry based on a brief 5-second grace period. It should retrieve the newly generated tokens from a temporary Redis cache and resend them to the client to ensure a seamless user experience.
- **B)** The server must treat this as a potential token theft (a replay attack). It must immediately deny the request, and critically, it must actively revoke the *newly issued* Refresh Token and all other tokens associated with that user's session, forcing them to re-authenticate with their password.
- **C)** The server should return a 429 Too Many Requests status, instructing the client to back off. Refresh tokens are cryptographic and cannot be "invalidated" server-side without maintaining state, which violates REST principles.
- **D)** The server should accept the old refresh token but issue an Access Token with a highly restricted scope (e.g., read-only access) until the user's IP address stabilizes.

Answer: B

Explanation: This is the core principle of Refresh Token Rotation and "Reuse Detection". If a refresh token that has already been used is presented again, the server cannot know if it's an innocent network retry or an attacker who stole the old token. The only secure

response is to assume a breach has occurred and sever the entire trust chain by revoking the *currently active* tokens associated with that original grant.

Question 19

You are building an inventory management system. Your Product entity has a quantity field. Admin A loads the product page (quantity = 100). Admin B loads the exact same page (quantity = 100). Admin A processes a shipment, updating the quantity to 150, and clicks save. Admin B processes a return, updating the quantity to 110, and clicks save 5 seconds later. Because Admin B was looking at stale data, their save overwrites Admin A's changes, leaving the database with 110 instead of the correct 160.

What is the standard Spring Data JPA mechanism to prevent this "Lost Update" anomaly without resorting to heavy database-level table locks?

- **A)** Change the transaction isolation level on the service method to `@Transactional(isolation = Isolation.SERIALIZABLE)`. This forces all database interactions to occur strictly sequentially.
- **B)** Add an integer field annotated with `@Version` to the Product entity. Hibernate will automatically increment this version number on every update. If Admin B tries to save with an outdated version number, Spring will throw an `ObjectOptimisticLockingFailureException`.
- **C)** Annotate the `saveProduct` method with `@Lock(LockModeType.PESSIMISTIC_WRITE)`. This tells Spring Data to issue a `SELECT ... FOR UPDATE` SQL command, blocking any other threads from reading the row until the transaction is complete.
- **D)** Implement a custom save method using `@Modifying` and `@Query("UPDATE Product p SET p.quantity = p.quantity + :change WHERE p.id = :id")` to force the calculation to happen atomically inside the database engine rather than in application memory.

Answer: B

Explanation: This scenario perfectly describes the "Lost Update" problem. Options A and C use Pessimistic Locking, which hurts performance by blocking database rows. Option B utilizes Optimistic Locking. By adding an `@Version` field, Hibernate includes `WHERE id = ? AND version = ?` in its update statements. If the version doesn't match, it means someone else modified the data first, safely aborting the transaction and throwing an exception.

Question 20

You are architecting a new microservice. You have an `OrderService` and a `PaymentService`. The `OrderService` needs to call `paymentService.processPayment()` when an order is finalized. The `PaymentService` needs to call `orderService.updateOrderStatus()` if a payment asynchronously clears later. You define both beans using constructor injection:

```
public OrderService(PaymentService ps) { ... }
```

```
public PaymentService(OrderService os) { ... }
```

When you start the Spring Boot application, it crashes immediately with a `BeanCurrentlyInCreationException`.

As of Spring Boot 2.6+, how does the framework treat this scenario, and what is the architecturally recommended way to resolve it?

- **A)** Spring Boot automatically detects circular dependencies and attempts to create a CGLIB proxy for one of the beans. The crash means your classes are marked as final, preventing proxy generation. Remove the final keyword.
- **B)** Spring Boot prohibits circular dependencies by default as they indicate a major architectural flaw (tight coupling). The recommended fix is to redesign the code—for example, by introducing an event-driven approach (Spring Events) or extracting the shared logic into a third service (e.g., OrderFulfillmentService).
- **C)** Constructor injection cannot resolve circular dependencies because the objects must be fully instantiated before passing them. The recommended fix is to switch both classes to field injection using `@Autowired` directly on the private variables, which delays initialization.
- **D)** You must add the `@Lazy` annotation to the `@Service` declaration on both classes. This tells the application context to delay the instantiation of both beans until the first HTTP request hits the controller.

Answer: B

Explanation: Starting in Spring Boot 2.6, circular dependencies are explicitly disabled by default. If you have a circular dependency, it is almost always a sign of a design smell (a violation of the Single Responsibility Principle). The architectural fix is not to trick the framework into loading them, but to redesign the relationship, often by extracting the interdependent logic into a new, lower-level service or using messaging/events to decouple them.

Question 21

Your frontend team (building a React app on localhost:3000) is trying to consume a new Spring Boot REST API hosted on localhost:8080. You have added the `@CrossOrigin(origins = "http://localhost:3000")` annotation directly to your `@RestController`. However, the frontend developers complain that their browser console is throwing a "CORS Policy: No Access-Control-Allow-Origin header is present" error. When you check the server logs, you see that the browser's preflight OPTIONS request is returning an HTTP 401 Unauthorized status before the CORS headers can even be evaluated.

What is the architectural reason for this failure, and how must you configure Spring Boot to resolve it?

- **A)** The `@CrossOrigin` annotation only applies to GET and POST requests by default. You must explicitly add `methods = {RequestMethod.OPTIONS}` to the annotation to allow preflight checks.
- **B)** Spring Security intercepts the request before it reaches the MVC Controller. Because the OPTIONS request does not contain a JWT or session cookie, Spring Security rejects it with a 401. The fix is to add `.requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()` to your `SecurityFilterChain`.
- **C)** Browsers do not send the Origin header during local development for security reasons, causing the Spring CORS filter to reject the request. The fix is to configure a proxy proxy inside the React application's `package.json` to spoof the origin.
- **D)** The `@CrossOrigin` annotation is incompatible with `@RestController`. It can only be used on classes annotated with `@Controller` that return view resolvers. You must implement a global `WebMvcConfigurer` instead.

Answer: B

Explanation: This is a classic integration issue between Spring Web MVC and Spring Security. The `@CrossOrigin` annotation works at the MVC level (after the request passes

the servlet filters). However, Spring Security operates in the Filter Chain *before* MVC. When the browser sends the preflight OPTIONS request (which never contains credentials), Spring Security blocks it because it isn't authenticated. You must explicitly configure Spring Security to let OPTIONS requests pass through so the CORS processor can handle them.

Question 22

You are working on a monolithic Spring Boot application and decide to implement method-level security. You enable it globally and add `@PreAuthorize("hasRole('ADMIN')")` to a highly sensitive `deleteUserAccount(Long id)` method inside your `UserService`.

Later, you create a new public method in the *same* `UserService` class called `cleanupInactiveAccounts()`. Inside this new method, you write a loop that calls `this.deleteUserAccount(id)` for several users.

During testing, an unauthenticated user triggers the `cleanupInactiveAccounts()` method via an exposed API endpoint. To your shock, the accounts are deleted successfully without throwing an `AccessDeniedException`.

Why did the `@PreAuthorize` annotation fail to protect the sensitive method?

- **A)** `@PreAuthorize` uses Spring AOP (Aspect-Oriented Programming). AOP creates a proxy around the bean. When a method calls another method within the *same* instance using `this.`, it bypasses the proxy entirely, meaning the security checks are never intercepted or executed.
- **B)** Spring Security's `@PreAuthorize` is strictly bound to the HTTP Request context. Because `cleanupInactiveAccounts()` was the initial entry point, its "permitAll" security context was permanently cascaded to all subsequent downstream method calls.
- **C)** Method-level security annotations like `@PreAuthorize` only work on interfaces. If `UserService` is a concrete class that does not implement an interface, Spring silently ignores the annotation during bean initialization.
- **D)** The `hasRole('ADMIN')` expression requires the user's authority string to be exactly "ADMIN". If the database stores it as "ROLE_ADMIN", the expression evaluates to true by default to prevent system lockouts during misconfigurations.

Answer: A

Explanation: This is known as the "AOP Self-Invocation" problem. Spring applies cross-cutting concerns (like Security, Transactions, and Caching) by wrapping your beans in dynamic proxies. The proxy intercepts external calls to apply the logic. However, if a method inside the class calls another method in the same class (using `this`), the call happens directly on the target object, completely bypassing the proxy and its security checks.

Question 23

In your e-commerce platform, the `OrderService` saves a new order to the database and immediately publishes an `OrderCreatedEvent` using `ApplicationEventPublisher`. You have a separate `EmailNotificationListener` that listens for this event and sends a confirmation email to the customer.

Occasionally, a database constraint violation (like a duplicate transaction ID) occurs at the very end of the service method, causing the `@Transactional` method to roll back the order.

However, customers are still receiving the "Order Confirmed" emails for these failed, rolled-back orders.

How do you guarantee that the email is strictly sent *only* if the database transaction commits successfully?

- **A)** Move the `publishEvent()` call to a `finally` block at the end of the `OrderService` method and manually check `TransactionAspectSupport.currentTransactionStatus().isRollbackOnly()` before publishing.
- **B)** Change the `@EventListener` annotation in the `EmailNotificationListener` to `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`. This ensures the event is only dispatched to this specific listener after the current transaction successfully commits.
- **C)** Inject the `EntityManager` into the listener and manually query the database for the order ID. If the order does not exist, throw an `IllegalStateException` to abort the email sending process.
- **D)** Annotate the `OrderService` method with `@Transactional(isolation = Isolation.READ_COMMITTED)`. This will place a temporary lock on the event bus until the database releases its row locks.

Answer: B

Explanation: Standard `@EventListener` executes immediately when the event is published, regardless of the transaction's outcome. If you are doing actions that cannot be rolled back (like sending an email or calling an external API), you must wait to ensure the database actually committed. Spring provides `@TransactionalEventListener` specifically for this architectural pattern, allowing you to bind the listener's execution to a specific phase of the transaction lifecycle (like `AFTER_COMMIT`).

Question 24

Your batch processing service uses Spring Data JPA to migrate 500,000 records from a CSV file into a relational database. Your method is annotated with `@Transactional`. It reads the CSV, creates `Customer` entity objects, and calls `customerRepository.save(customer)` inside a loop.

After successfully processing about 80,000 records, the application suddenly crashes with a `java.lang.OutOfMemoryError: Java heap space`.

What is the fundamental JPA mechanism causing this memory leak during the batch insert, and how do you resolve it?

- **A)** `save()` is a blocking operation. The connection pool runs out of available threads to handle the 80,000 concurrent database locks, causing the JVM to allocate massive amounts of memory for thread waiting states. The fix is to use `saveAll()`.
- **B)** The `Customer` entities contain a `@Lob` (Large Object) field for a profile picture. Hibernate automatically eager-fetches LOBs into memory during batch inserts to verify checksums. The fix is to mark the `@Lob` as `fetch = FetchType.LAZY`.
- **C)** Within a single transaction, the Hibernate Session (First-Level Cache) keeps a strong memory reference to every single entity that has been saved. Because the transaction hasn't committed yet, the Garbage Collector cannot clean them up. The fix is to periodically call `flush()` and `clear()` on the `EntityManager` inside the loop.
- **D)** Spring Boot's internal HikariCP connection pool keeps a history of all executed SQL statements in memory for the Actuator `/metrics` endpoint. The fix is to disable statement caching in the `application.yml` file.

Answer: C

Explanation: The First-Level Cache is bound to the Hibernate Session (which corresponds to the @Transactional boundary). Every entity you pass to save() becomes "managed" and is cached in memory until the transaction ends. If you process hundreds of thousands of records in a single transaction, the cache fills up the JVM heap and causes an OOM error. To do batch processing with JPA, you must regularly push the changes to the DB (flush()) and then detach the entities from the cache (clear()) so the Garbage Collector can reclaim the memory.

Question 25

You are building an internal custom Spring Boot Starter (e.g., company-security-starter) that all microservices in your organization will include in their pom.xml. Your starter auto-configures a central JwtValidationService bean.

However, for one specific legacy microservice, the team needs to write their own highly customized JwtValidationService to handle old token formats, but they still want to use the rest of your starter's features. When they define their custom bean, Spring Boot throws a NoUniqueBeanDefinitionException at startup because it finds both your auto-configured bean and their custom bean.

Which Spring Boot annotation must you add to the bean declaration *inside your starter library* to allow applications to seamlessly override it?

- **A)** @ConditionalOnProperty(name = "jwt.validation.override", havingValue = "true")
- **B)** @Primary
- **C)** @ConditionalOnMissingBean
- **D)** @Qualifier("defaultJwtValidator")

Answer: C

Explanation: This is the cornerstone of Spring Boot Auto-Configuration. When you build a starter, you want to provide sensible defaults that back away gracefully if the consumer provides their own implementation. By annotating your library's bean with @ConditionalOnMissingBean, you tell Spring: "Only instantiate this bean if the developer hasn't already defined a bean of this type in their own application context." Option B (@Primary) would force the developer to modify their code, rather than the library intelligently backing down.

Question 26

Your application relies on an external REST API that requires over 40 different configuration parameters (URLs, timeouts, retry counts, static API keys). A junior developer adds all 40 keys to the application.yml file and then injects them into the ExternalApiClient class using 40 individual @Value("\${api.url}"), @Value("\${api.timeout}"), etc., annotations.

During a code review, you point out that this is highly prone to typos, provides no IDE auto-completion for the YAML file, and makes validation incredibly difficult.

What is the best-practice Spring Boot mechanism to refactor this configuration into a type-safe, easily validated structure?

- **A)** Create a custom `EnvironmentPostProcessor` class that reads the YAML file using Jackson during startup and maps it into a Singleton `Map<String, Object>` that can be queried dynamically.
- **B)** Create a POJO (Plain Old Java Object) class, annotate it with `@ConfigurationProperties(prefix = "api")`, and declare fields matching the YAML structure. Spring will automatically bind the YAML properties to this object, enabling strong typing and standard Bean Validation (`@NotNull`, `@Min`).
- **C)** Consolidate the 40 properties into a single comma-separated string in the YAML file. Inject it using one `@Value` annotation into a `List<String>`, and parse the values by index position in the constructor.
- **D)** Move the configuration out of the YAML file and into an XML file. Use Spring's legacy `@ImportResource` to strictly validate the configuration parameters against an XSD schema at compile time.

Answer: B

Explanation: Injecting massive amounts of configuration via `@Value` is an anti-pattern. `@ConfigurationProperties` is Spring Boot's powerful feature for hierarchical, type-safe property binding. It automatically maps properties with a specific prefix from your YAML/properties file into a Java object. This provides immense benefits: you get IDE autocomplete in your YAML files, you can use JSR-303 validation annotations directly on the fields, and the code is drastically cleaner.

Question 27

You retrieve a `User` entity from the database using JPA. The transaction closes, and you pass this `User` object to the presentation layer. The user modifies their email address in a web form, and the updated `User` object is passed back to your Service layer.

You write the following code in your Service:

```
user.setEmail(newEmail);
```

You notice that the database is never updated with the new email, even though no exceptions are thrown.

Why did JPA fail to execute the `UPDATE` SQL statement, and what must be done to fix it?

- **A)** The entity is in a "Detached" state because the original transaction closed. JPA's "dirty checking" mechanism only monitors "Managed" entities attached to an active Persistence Context. You must pass the object back to the repository using `repository.save(user)` to reattach it and trigger the update.
- **B)** JPA entities are strictly immutable by design. The `setEmail` method modifies the object reference in memory, breaking the cryptographic hash that Hibernate uses to track changes. You must instantiate a completely new `User` object and copy the properties over.
- **C)** The connection pool timed out while the user was filling out the web form. You must configure `@Transactional(timeout = 3600)` on the controller to keep the database connection alive during UI interactions.
- **D)** The `@Column(updatable = false)` annotation is applied by default to all String fields in Spring Data JPA to prevent SQL injection. You must explicitly set it to true in the entity mapping.

Answer: A

Explanation: Understanding Entity Lifecycle states (Transient, Managed, Detached, Removed) is critical in JPA. When an entity is loaded, it is Managed. When the transaction ends, the Persistence Context is closed, and the entity becomes Detached. Hibernate does not track changes to Detached entities. To save changes made to a Detached entity, you must reattach it (usually via `merge()`, which Spring Data's `save()` method handles automatically under the hood for existing entities).

Question 28

A junior developer asks you for help with a Core Java issue. They have a microservice that aggregates data by calling three separate, slow, external REST APIs.

Currently, the code makes the HTTP calls sequentially:

```
String data1 = api1.getData(); (takes 3 seconds)
```

```
String data2 = api2.getData(); (takes 3 seconds)
```

```
String data3 = api3.getData(); (takes 3 seconds)
```

The total response time for this method is consistently over 9 seconds. The developer wants to execute them in parallel so the total time is closer to 3 seconds, but they need to wait until *all three* responses arrive before combining them into a final JSON response.

Which modern Core Java feature provides the cleanest, non-blocking way to achieve this parallel aggregation?

- **A)** Instantiate three standard `java.lang.Thread` objects, pass in `Runnable` tasks, and call `.start()` on all three. Then use a `while(true)` loop to continually poll the threads until `thread.isAlive()` returns false for all of them.
- **B)** Wrap each API call in a `CompletableFuture.supplyAsync()`. Then use `CompletableFuture.allOf(future1, future2, future3).join()` to wait for all of them to complete simultaneously before extracting their results.
- **C)** Add the `synchronized` keyword to the method signature of the external API client. This instructs the JVM thread scheduler to interleave the execution of the network I/O streams automatically.
- **D)** Use the `java.util.Timer` class to schedule the three tasks to execute 1 millisecond apart. Attach a `PropertyChangeListener` to the timer that aggregates the variables when the timer reaches zero.

Answer: B

Explanation: Introduced in Java 8, `CompletableFuture` is the modern, robust API for asynchronous programming. `supplyAsync()` offloads the task to a worker thread pool. `CompletableFuture.allOf()` is a static method specifically designed to take an array of futures and return a single new future that completes only when *every* provided future has completed, perfectly solving the "parallel fetch and aggregate" scenario without blocking the main thread prematurely or using inefficient polling loops.

Question 29

During an interview for a Senior Backend Developer role, the interviewer writes the following Core Java code on the whiteboard:

Java

```
String s1 = "Trivandrum";
```

```
String s2 = "Trivandrum";
```

```
String s3 = new String("Trivandrum");
```

```
System.out.println(s1 == s2);
```

```
System.out.println(s1 == s3);
```

```
System.out.println(s1.equals(s3));
```

What will be the exact console output of this code, and what underlying JVM memory concept dictates the result of the second print statement?

- **A)** true, true, true. The JVM optimizes memory by running a background garbage collection thread that collapses all strings with identical character sequences into a single memory address, regardless of how they are instantiated.
- **B)** true, false, true. The second statement is false because the new keyword forces the JVM to create a brand new object in the standard Heap memory, bypassing the String Constant Pool where s1 and s2 reside. == compares memory addresses, not the content.
- **C)** false, false, true. Strings in Java are immutable. Therefore, s1 and s2 are completely different objects in memory. The == operator always returns false when comparing objects, making .equals() strictly required for all comparisons.
- **D)** false, true, false. The String Pool only caches objects created with the new keyword. Literal strings like s1 are stored on the execution Stack and are destroyed immediately after the method exits, causing unpredictable memory references.

Answer: B

Explanation: This tests knowledge of the String Constant Pool and Object References. When you create a string literal (String s1 = "Trivandrum"), the JVM places it in a special memory area called the String Pool. If you create another literal with the same value (s2), the JVM reuses the exact same memory reference (so s1 == s2 is true). However, using the new keyword (s3) forces the JVM to bypass the pool and create a new object in the general heap memory. Because == checks if memory addresses are identical, s1 == s3 is false. .equals() correctly checks the actual character sequence, returning true.

Question 30

You are designing a REST API in Spring MVC. You have a controller method mapped to /api/documents.

A legacy mobile application expects the response to be formatted as XML.

A new web frontend expects the response to be formatted as JSON.

You want to use the exact same controller method to serve both clients without duplicating your data-fetching logic or writing complex if/else statements inspecting the headers.

How does Spring Boot natively handle this requirement (Content Negotiation), and what must the client provide to trigger it?

- **A)** Spring Boot uses the User-Agent HTTP header. If the header contains "Mozilla" or "Chrome", Spring automatically converts the response to JSON. If it contains "Android" or "iOS", it converts it to XML.
- **B)** Spring Boot relies on Content Negotiation based on the Accept HTTP header sent by the client (e.g., Accept: application/json or Accept: application/xml). As long as the Jackson XML and JSON libraries are on the classpath, Spring's `HttpMessageConverters` will automatically serialize the returned Java object into the requested format.
- **C)** Content Negotiation requires defining multiple endpoints with the exact same path but different method names. You must annotate one with `@GetMapping(produces = "json")` and the other with `@GetMapping(produces = "xml")`. Spring intercepts the request and routes it based on network latency.
- **D)** You must change the method signature to return a `raw byte[]`. Then, implement a Filter that intercepts the byte stream, detects the origin IP address, and uses a hardcoded configuration map to serialize the bytes into either XML or JSON before dispatching the response.

Answer: B

Explanation: Spring MVC's Content Negotiation is powerful and elegant. By default, it looks at the Accept header sent by the HTTP client. The controller method simply returns a standard Java object (e.g., `ResponseEntity<DocumentDTO>`). Before the response goes out, Spring's `HttpMessageConverters` step in. If the client asked for XML (and the `jackson-dataformat-xml` dependency is present), it converts the object to XML. If they asked for JSON, it converts it to JSON. The developer writes the business logic once.

Question 31

Your development team complains that the Spring Boot integration test suite execution time has ballooned from 2 minutes to over 45 minutes. You investigate and notice that almost every test class in the suite is annotated with `@SpringBootTest` and utilizes `@MockBean` to mock out various external service calls. CPU and memory usage during the test run are within normal limits, but the logs show the Spring Application Context starting up and shutting down dozens of times.

What is the underlying mechanism causing this massive performance degradation in the test suite, and how should it be architected instead?

- **A)** `@MockBean` utilizes CGLIB to create proxy classes at runtime. The JVM's Metaspace is filling up with these generated classes, triggering aggressive, blocking Garbage Collection pauses. The fix is to increase `-XX:MaxMetaspaceSize` in the Maven Surefire plugin.
- **B)** Spring's `TestContext` framework heavily caches the Application Context. However, introducing a `@MockBean` alters the context signature for that specific test class. This forces Spring to invalidate the cache and completely rebuild a brand-new Application Context for every test class that uses a different set of mocks. The fix is to use standard Mockito `@Mock` with constructor injection where possible, or consolidate context configurations.
- **C)** The `@SpringBootTest` annotation binds to a random available port for each test class to avoid conflicts. The operating system is running out of ephemeral ports and the tests are being artificially throttled while waiting for `TIME_WAIT` sockets to close. The fix is to use `webEnvironment = WebEnvironment.MOCK`.
- **D)** `@MockBean` injections are not thread-safe. Maven Surefire runs tests in parallel by default, causing deadlocks inside Spring's internal bean registry. The suite is constantly waiting for internal lock timeouts. The fix is to configure Surefire to run tests sequentially.

Answer: B

Explanation: Spring Boot's testing framework is designed to cache and reuse the Application Context across multiple tests to keep execution times fast. However, if a test class modifies the context (e.g., by replacing a real bean with a `@MockBean`), Spring must create a completely separate context for that test. If 50 test classes use different `@MockBean` setups, Spring boots the entire application 50 times. Using standard `@Mock` and isolated unit tests (or standardizing integration test configurations) is the correct architectural fix.

Question 32

You are developing a high-throughput Spring MVC application running on Apache Tomcat. To track user activity across various service layers without passing the user ID through every method signature, a developer implements a `ThreadLocal<String> currentUser`. The ID is set at the beginning of the controller method and read deep within the data access layer.

In production, users begin seeing sensitive data and dashboard metrics belonging to other random users who are logged in at the same time.

What is the root cause of this data bleeding, and what is the mandatory Core Java / Spring lifecycle fix?

- **A)** `ThreadLocal` variables are inherently synchronized, causing massive thread contention under load. The JVM's context switcher is misaligning the memory pointers to avoid deadlocks. The fix is to replace `ThreadLocal` with a `ConcurrentHashMap`.
- **B)** Tomcat utilizes a fixed pool of worker threads to handle HTTP requests. When a request finishes, the thread is not destroyed; it is returned to the pool to serve the next user. If the `ThreadLocal` is not explicitly cleared, the next user assigned to that thread inherits the previous user's data. The fix is to clear the variable in a finally block or a `HandlerInterceptor.afterCompletion` method.
- **C)** Spring Controllers are singletons. A `ThreadLocal` defined inside a singleton bean automatically escalates to an application-scoped variable, bypassing thread isolation. The fix is to change the controller's scope to `@Scope("request")`.
- **D)** The application is running on a multi-core processor. CPU L3 cache is syncing the `ThreadLocal` variables across different physical cores, leading to race conditions. The fix is to declare the `ThreadLocal` variable with the `volatile` keyword.

Answer: B

Explanation: This is a critical security vulnerability caused by misunderstanding thread pools. Web servers like Tomcat use thread pools. A thread processes Request A, finishes, and is recycled to process Request B. If Request A puts data into a `ThreadLocal` and fails to remove it, Request B will read Request A's data when it accesses the `ThreadLocal`. You must always meticulously clean up `ThreadLocal` variables in a finally block or a global interceptor when the execution flow completes.

Question 33

Your Spring Boot application handles complex financial transfers. The main `@Transactional` method, `executeTransfer()`, performs several database updates. Before returning, it calls an `AuditService.logTransferAttempt()` method, which is also annotated with `@Transactional`.

If a user attempts a transfer with insufficient funds, the `executeTransfer()` method throws an `InsufficientFundsException` (a `RuntimeException`), which correctly triggers a rollback of the

financial updates. However, the compliance team reports that the audit logs for these failed attempts are entirely missing from the database.

How must you configure the transactional boundaries to ensure the audit log is saved permanently, even if the parent transfer transaction rolls back?

- **A)** You must annotate the parent `executeTransfer` method with `@Transactional(noRollbackFor = InsufficientFundsException.class)` so that the exception does not trigger a global rollback, allowing both the transaction and the audit log to commit.
- **B)** You must add the `@Async` annotation to the `logTransferAttempt` method. This offloads the database write to a separate thread, which automatically bypasses the transactional boundaries of the main HTTP request thread.
- **C)** You must change the propagation behavior on the `logTransferAttempt` method to `@Transactional(propagation = Propagation.REQUIRES_NEW)`. This suspends the outer transaction, starts a completely new, independent transaction for the audit log, commits it, and then resumes the outer transaction.
- **D)** You must catch the exception inside `executeTransfer()`, manually call `entityManager.flush()`, and then rethrow the exception. This forces the database to write the audit log to disk before the proxy initiates the rollback phase.

Answer: C

Explanation: By default, Spring uses `Propagation.REQUIRED`. This means the inner method (`logTransferAttempt`) joins the existing outer transaction. If the outer method throws an exception and rolls back, the *entire* single transaction rolls back, including the audit log. To guarantee the audit log commits regardless of the parent's fate, it must execute in its own isolated transaction boundary using `Propagation.REQUIRES_NEW`.

Question 34

Your Spring MVC REST API acts as a gateway, querying an extremely slow external third-party SOAP service that takes 5 seconds to respond. Your Tomcat server is configured with a maximum of 200 worker threads. During peak traffic, your application receives 300 concurrent requests per second. The server quickly becomes completely unresponsive, returning 503 Service Unavailable. However, server monitoring shows that CPU utilization is below 10% and memory usage is stable.

What Spring MVC architectural pattern should you implement to prevent thread pool exhaustion and keep the application responsive while waiting for the slow external service?

- **A)** Increase the `server.tomcat.threads.max` property in `application.yml` from 200 to 10,000. Tomcat threads are extremely lightweight, and modern operating systems can easily handle tens of thousands of idle threads waiting on network I/O.
- **B)** Change the controller method's return type from `ResponseEntity<String>` to `CompletableFuture<ResponseEntity<String>>` or `DeferredResult`. Execute the slow external call in a separate custom thread pool. This releases the Tomcat worker thread immediately, allowing it to serve other incoming requests while the background thread waits for the SOAP response.
- **C)** Wrap the external API call in a `Resilience4j` or Spring Retry `@Retryable` annotation. If the thread pool is exhausted, the framework will gracefully queue the requests and retry them exponentially without blocking the primary socket listeners.
- **D)** Migrate the external SOAP call to a `@Scheduled` background cron job that caches the third-party data in a local Redis instance every minute, so the controller never has to make an outbound network call.

Answer: B

Explanation: This scenario describes classic Thread Pool Exhaustion due to blocking I/O. 200 requests come in, 200 Tomcat threads get stuck waiting for 5 seconds for the external API. The 201st request is rejected, even though the CPU is doing absolutely nothing (just waiting). Returning a `CompletableFuture` or `DeferredResult` utilizes Spring's asynchronous request processing. It frees up the Tomcat worker thread to handle new requests, while a separate background thread waits for the slow API. Once the API returns, the result is dispatched back to the client.

Question 35

You upgrade a legacy Spring Boot 2.x application to Spring Boot 3.x. The application exposes a purely stateless REST API secured via JWTs.

After deploying the upgraded code, all GET requests work perfectly. However, frontend developers report that every single POST, PUT, and DELETE request is suddenly returning a 403 Forbidden status code. You verify that the JWTs are perfectly valid, not expired, and contain the correct roles.

What default security mechanism, newly strictly enforced or changed in the upgrade, is causing this selective HTTP method blocking, and how do you resolve it for a stateless API?

- **A)** Cross-Origin Resource Sharing (CORS) policies are strictly enforced on mutating HTTP methods in Boot 3.x. You must add `@CrossOrigin` to all POST, PUT, and DELETE mappings.
- **B)** Spring Security 6.x automatically maps POST/PUT/DELETE methods to require the `ROLE_ADMIN` authority by default unless explicitly overridden in the filter chain. You must update the users' JWTs to include this role.
- **C)** Cross-Site Request Forgery (CSRF) protection is enabled by default. CSRF intercepts state-changing methods (POST, PUT, DELETE) to check for a specific token. Because this is a stateless API using JWTs (not session cookies), CSRF is unnecessary and is blocking the requests. The fix is to explicitly disable it via `http.csrf(AbstractHttpConfigurer::disable)`.
- **D)** The new HTTP Strict Transport Security (HSTS) defaults require all mutating requests to be upgraded to HTTPS. Since the frontend is likely testing on `http://localhost`, the framework drops the payloads and returns a 403.

Answer: C

Explanation: CSRF (Cross-Site Request Forgery) protection is enabled by default in Spring Security. It specifically targets state-changing HTTP operations (POST, PUT, DELETE) by requiring a CSRF token. However, CSRF attacks rely on the browser automatically attaching session cookies. If your API is purely stateless and relies on the client manually attaching a JWT in the Authorization header, CSRF attacks are impossible by design. Therefore, the CSRF protection is unnecessary overhead that breaks the API unless explicitly disabled.

Question 36

You are optimizing a Core Java batch processor that calculates end-of-day balances for 1 million accounts. You change the standard loop to use the Streams API:
`accountList.parallelStream().forEach(account -> ... )`.

Inside the lambda expression, you do the calculation and add the result to a globally scoped `ArrayList<BalanceDTO> results = new ArrayList<>()`.

When you run the process, the application frequently crashes with an `ArrayIndexOutOfBoundsException` or `ConcurrentModificationException` deep inside the `java.util.ArrayList` source code, even though you are only adding items, not removing them.

Why is the `parallelStream` causing this specific internal collection failure, and what is the idiomatic Streams API fix?

- **A)** `parallelStream()` splits the workload across multiple threads using the ForkJoin pool. `ArrayList` is not thread-safe; concurrent `.add()` operations cause race conditions where threads overwrite internal array indices or trigger a resize simultaneously, corrupting the array structure. The fix is to return values from the stream and use `.collect(Collectors.toList())`.
- **B)** The ForkJoin pool used by `parallelStream` has a default timeout of 60 seconds per thread. If the calculation takes too long, the thread is forcefully terminated, leaving the `ArrayList` in a mathematically corrupted state. The fix is to use a standard for loop with a custom `ThreadPoolExecutor`.
- **C)** `ArrayList` has a hard limit of 65,535 elements when accessed by virtual threads in modern Java. Because you have 1 million accounts, it overflows the index capacity. The fix is to switch the collection to a `LinkedList`.
- **D)** `parallelStream` executes in reverse order to optimize memory caching. The `ArrayList` is attempting to insert elements at index 1,000,000 before index 0 has been initialized, causing an bounds exception. The fix is to pre-allocate the `ArrayList` with `new ArrayList<>(1000000)`.

Answer: A

Explanation: This is a classic violation of parallel stream design principles. Parallel streams execute operations concurrently across multiple threads. If the operation inside a parallel stream has side-effects—like modifying a shared, non-thread-safe collection (`ArrayList`)—it will cause race conditions and data corruption. `ArrayList.add()` is not an atomic operation. The correct, idiomatic approach in functional programming is to avoid side-effects entirely by letting the stream aggregate the results itself using a reduction operation like `.collect(Collectors.toList())`.

Question 37

A production PL/SQL database is experiencing severe lockups. You analyze the Oracle/PostgreSQL trace logs and find two transactions constantly colliding.

Transaction A executes: `UPDATE EMPLOYEES SET salary = ... WHERE id = 10;` followed by `UPDATE DEPARTMENTS SET budget = ... WHERE id = 5;`

Transaction B executes: `UPDATE DEPARTMENTS SET budget = ... WHERE id = 5;` followed by `UPDATE EMPLOYEES SET salary = ... WHERE id = 10;`

The database engine detects the issue and automatically terminates one of the transactions, throwing an `ORA-00060: deadlock detected while waiting for resource`.

What is the architectural cause of this deadlock, and what is the standard software engineering practice to prevent it?

- **A)** The transactions are using Optimistic Locking, which causes a race condition when the version numbers overlap during high concurrency. The fix is to switch the application to Pessimistic Locking using `SELECT ... FOR UPDATE`.

- **B)** This is a classic circular wait deadlock. Trans A holds a lock on EMPLOYEES and waits for DEPARTMENTS. Trans B holds a lock on DEPARTMENTS and waits for EMPLOYEES. Neither can proceed. The fix is to enforce a strict, application-wide ordering rule: whenever multiple tables must be updated in a transaction, they must always be locked/updated in the exact same alphabetical or hierarchical order.
- **C)** The database isolation level is set too high (Serializable). This prevents phantom reads but causes entire tables to be locked instead of individual rows. The fix is to lower the isolation level to Read Committed.
- **D)** The UPDATE statements are missing the COMMIT keyword after each line. Because the transaction stays open too long, the database's internal lock manager runs out of memory and kills the session. The fix is to commit immediately after every single DML statement.

Answer: B

Explanation: This scenario outlines the textbook definition of a database deadlock (a circular wait). Two transactions acquire exclusive row locks on different resources and then attempt to acquire locks on the resources held by the other. The database must kill one to break the cycle. The universally accepted architectural solution is resource ordering: if all transactions across the entire application always update tables in the same predefined order (e.g., always EMPLOYEES first, then DEPARTMENTS), a circular wait becomes mathematically impossible.

Question 38

Your Spring Boot microservices run in Kubernetes. You use Spring Boot Actuator for health checks (/actuator/health), which Kubernetes probes to determine if a pod is healthy.

Your application relies on a legacy Oracle database that occasionally drops connections for 2-3 seconds during network blips. Your application is highly resilient; it uses a local Redis cache to serve read requests when the DB is temporarily down.

However, during these brief DB blips, the Actuator health endpoint immediately reports "status": "DOWN". Kubernetes sees this, assumes the pod has crashed, and ruthlessly restarts all your pods, turning a 3-second database glitch into a 5-minute total system outage.

How do you instruct Spring Boot Actuator to ignore the database status so Kubernetes doesn't restart the resilient pods?

- **A)** Disable the default database health check entirely by setting management.health.db.enabled=false in the application.yml. Kubernetes will now only verify that the Spring Boot web server is responding to HTTP requests.
- **B)** Annotate your core DataSource configuration bean with @Lazy. This instructs Actuator to only check the database health when a user actively executes a SQL query, rather than during background pinging.
- **C)** Change the Kubernetes livenessProbe to point to /actuator/metrics instead of /actuator/health. The metrics endpoint never returns a 503 error, ensuring the pod stays alive.
- **D)** Implement a custom Spring ApplicationListener that intercepts the AvailabilityChangeEvent. When the DB goes down, publish a ReadinessState.ACCEPTING_TRAFFIC event to forcefully override the Actuator status.

Answer: A

Explanation: Spring Boot Actuator's /health endpoint automatically aggregates the health of all detected auto-configured dependencies (Database, Redis, RabbitMQ, etc.). If *any* of them report as DOWN, the overall application status becomes DOWN, triggering a 503 HTTP status. If your application is designed to survive a database outage gracefully, the database's health should not dictate the pod's liveness. The simplest and most standard fix is to disable the auto-configured DB health indicator via properties, or write a custom HealthIndicator that reports the DB failure as a non-fatal warning instead of a DOWN state.

Question 39

You are designing a JPA entity User. You decide to override the Core Java equals() and hashCode() methods. To ensure uniqueness, you implement the methods using the database primary key: Long id; (which is annotated with @GeneratedValue(strategy = GenerationType.IDENTITY)).

In your service layer, you instantiate a new User, add it to a HashSet<User>, and then call userRepository.save(user);.

Later in the exact same method, you need to verify if the set contains the user, so you call set.contains(user);. It returns false, causing your business logic to fail, even though you are passing the exact same object reference.

Why did the HashSet fail to find the object after it was saved to the database?

- **A)** HashSet relies on the hashCode() method to determine which internal memory bucket to place the object in. Before save(), the ID was null (hashCode X). During save(), Hibernate assigned an ID (e.g., 5), which changed the object's hashCode() to Y. The contains() method looks in bucket Y, but the object is permanently lost in bucket X.
- **B)** When userRepository.save() executes, Spring Data JPA creates a completely new, detached clone of the object to prevent cache contamination. The contains() method is checking for the original transient object reference, which no longer matches the managed entity.
- **C)** The @GeneratedValue(strategy = GenerationType.IDENTITY) annotation forces Hibernate to execute the insert asynchronously. At the exact moment set.contains() is called, the database has not yet generated the ID, resulting in a NullPointerException swallowed by the Set API.
- **D)** Standard Java Collections are incompatible with JPA entity proxies. When the object was saved, Hibernate replaced it with a ByteBuddy proxy object. The HashSet is performing a strict class equivalence check (instanceof), which fails against the proxy.

Answer: A

Explanation: This highlights a critical rule when mixing JPA with Java Collections: never use a generated database ID in your equals() and hashCode() methods. If you put an entity into a Set before it is saved, its ID is null. After it is saved, the ID is populated, changing its internal state. A fundamental contract of Hash-based collections is that an object's hash code must *never* change while it is inside the collection. If it does, the collection can no longer locate the object. The best practice is to use a natural, immutable "business key" (like a UUID or email) for equality checks.

Question 40

Your team uses Spring's @Transactional heavily. You review a junior developer's code:

Java

@Service

```
public class OrderService {
```

```
@Transactional
```

```
public void processOrder() {
```

```
    saveOrderData();
```

```
    updateInventory();
```

```
}
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

```
public void updateInventory() {
```

```
    // updates stock, might throw OutOfStockException
```

```
}
```

```
}
```

The developer explains their intent: "When processOrder is called, it saves the order. Then it calls updateInventory. Because updateInventory has REQUIRES_NEW, it will run in a separate transaction. If inventory fails, I only want the inventory transaction to roll back, but the order data should still be saved."

During testing, when an OutOfStockException is thrown, the entire order is rolled back anyway.

Why did Spring ignore the REQUIRES_NEW directive and roll back the outer transaction?

- **A)** Propagation.REQUIRES_NEW is strictly designed for cross-database distributed transactions (XA Transactions). Because both methods interact with the same local database, Spring downgrades the propagation to REQUIRED to optimize connection pooling.
- **B)** The OutOfStockException is likely a checked exception. Spring declarative transaction management only triggers rollbacks for unchecked (RuntimeException) exceptions. It ignored the propagation because no valid rollback trigger was detected.
- **C)** The call to updateInventory() originates from *within* the same instance of the OrderService class. Spring's transactional behavior is powered by AOP proxies. Direct, internal method calls bypass the proxy object entirely, meaning the @Transactional annotation on the inner method is never intercepted or evaluated.
- **D)** The processOrder() method lacks a try-catch block. Even if the inner method spawned a new transaction, the exception bubbled up to the outer method, causing the outer transaction proxy to catch it and trigger a global rollback.

Answer: C

Explanation: This is the classic AOP Self-Invocation pitfall (similar to the Security proxy issue in Question 22). When an external class calls `orderService.processOrder()`, it hits the proxy, and a transaction begins. However, when `processOrder()` calls `updateInventory()`, it is using the implicit `this` reference. `this` refers to the target object, not the proxy. The proxy is completely bypassed, the `REQUIRES_NEW` annotation is never seen by the framework, and the code simply continues executing sequentially within the original, single transaction context.